

Model Card: Crypto Volatility Spike Prediction

Model Details

Model Name: crypto-volatility-random-forest

Model Type: Random Forest Classifier

Version: 1.0

Date: November 8, 2025 **Owner:** CMU Heinz - Operationalizing AI Course

Framework: scikit-learn

Model Description

A Random Forest classifier trained to predict cryptocurrency volatility spikes in real-time. The model identifies periods of abnormal price volatility (exceeding the 95th percentile threshold) using engineered features from streaming market data. This model is the final output of the `models/train.py` script.

Intended Use

Primary Use Case:

Real-time alerting system for cryptocurrency trading platforms to detect abnormal market volatility.

Intended Users:

- Cryptocurrency traders
- Risk management teams
- Algorithmic trading systems
- Market surveillance systems

Out-of-Scope Uses:

- Direct trading decisions without human oversight
- Use with cryptocurrencies other than Bitcoin (model trained on BTC/USD)
- Regulatory compliance (model not validated for regulatory purposes)

Training Data

Source: Kraken Exchange WebSocket API (simulated) **Symbol:** BTC/USD (Bitcoin/US Dollar)

Period: November 2025

Total Records: 32,233 observations

Train/Test Split: 80/20 (stratified by the notebook, loaded by `train.py`)

Training Set:

- Size: 25,789 samples
- Positive class (spikes): 1,290 samples (5.0%)

Test Set:

- Size: 6,448 samples

- Positive class (spikes): 322 samples (4.99%)

Feature Count: 13 engineered features

Label Definition

Volatility Spike:

Binary classification where 1 indicates a volatility spike (abnormal volatility) and 0 indicates normal market conditions.

Threshold (τ): 0.041799 (95th percentile of `volatility_60s`)

Train/Test Split Quality:

- An 80/20 stratified split was created by the `03_model_training.ipynb` notebook and saved to disk.
- The final `train.py` script uses these pre-split and validated datasets to ensure consistency.
- Data drift analysis was performed, and the report is available at reports/evidently_data_drift_report.html.

Features

Engineered Features (13 total)

Price Features (3):

- `price`: Current mid-market price (BTC/USD)
- `bid`: Best bid price
- `ask`: Best ask price

Volatility Features (3): ★ Most Important

- `volatility_30s`: 30-second rolling standard deviation
- `volatility_60s`: 60-second rolling standard deviation
- `volatility_120s`: 120-second rolling standard deviation

Return Features (3):

- `return_10s`: 10-second price return
- `return_30s`: 30-second price return
- `return_60s`: 60-second price return

Market Microstructure Features (4):

- `spread`: Bid-ask spread
- `intensity_30s`: 30-second trade intensity
- `intensity_60s`: 60-second trade intensity
- `intensity_120s`: 120-second trade intensity

Feature Importance (Top 5 from `train.py`)

Feature	Importance
---------	------------

Feature	Importance
volatility_30s	0.3128
volatility_60s	0.2860
volatility_120s	0.1904
price	0.0742
ask	0.0549

Key Insight: Volatility features consistently account for the majority of the model's predictive power.

Model Architecture

Algorithm: Random Forest Classifier

Hyperparameters:

- `n_estimators`: 100
- `max_depth`: 10
- `min_samples_split`: 20
- `min_samples_leaf`: 10
- `random_state`: 42

Performance

Quantitative Analysis (from `train.py`)

This table reflects the final performance metrics from the test set, as generated by the `models/train.py` script.

Model	Test F1	Test PR-AUC	Test Precision	Test Recall
Z-Score Baseline	0.9938	1.0000	0.9877	1.0000
Logistic Regression	0.7544	0.9870	1.0000	0.6056
Random Forest (Selected)	0.9984	1.0000	0.9969	1.0000

Model Selection Rationale: While the Z-Score baseline performed exceptionally well, the **Random Forest** model was selected for production. It demonstrates a slightly higher F1 score and, more importantly, its feature-based approach is more robust and adaptable to changing market dynamics compared to a simple, single-variable rule. It provides a stronger foundation for future improvements.

Inference Performance (from `infer.py --benchmark`)

This section validates that the model meets both functional and non-functional requirements.

Speed Performance (Real-Time Capability):

Metric	Value	Status
--------	-------	--------

Metric	Value	Status
Time per Sample	0.004ms	 Pass
Speedup vs. Real-Time	4271x faster	 Pass
Requirement	< 2x real-time	Met

Evaluation Metrics (Test Set Accuracy):

Metric	Value
F1 Score	0.9984
Precision	0.9969
Recall	1.0000
PR-AUC	1.0000

Conclusion: The model is extremely efficient and well-suited for a high-throughput, real-time streaming environment, while maintaining excellent predictive accuracy on the test set.

Ethical Considerations

Potential Biases

- **Temporal Bias:** Model trained on a specific, short time window in November 2025. Market dynamics will change.
- **Symbol Bias:** Only trained on BTC/USD; will not generalize to other assets without retraining.
- **Market Condition Bias:** The training data may not include "black swan" or extreme market events.

Limitations

1. **Recency Bias:** Assumes recent market patterns will persist.
2. **Single Asset:** Not tested on altcoins or other trading pairs.
3. **No Fundamental Data:** Ignores news, sentiment, and other external factors.

Risks

- **Overconfidence:** High precision may lead to over-reliance. The model is not infallible.
- **Adversarial Conditions:** The model has not been tested against intentionally manipulative market behavior.

Recommendations

Deployment Guidelines

1. **Review Configuration:** Check `config.yaml` for default parameters and `docs/feature_spec.md` for feature definitions.
2. **Set Environment Variables:** Use `.env` file for deployment-specific settings (Kafka servers, MLflow URI, API keys).

3. **Use as an Alerting System:** Predictions should trigger alerts for human review, not automated trades.
4. **Continuous Monitoring:** Implement drift detection on model predictions and input features (see `config.yaml` monitoring section).
5. **Regular Retraining:** Retrain the model weekly, or more frequently if market volatility increases.

Retraining Schedule

- **Frequency:** Weekly (configurable in `config.yaml` - `monitoring.model_monitoring.retrain_schedule`)
- **Triggers:**
 - Production F1 score drops below 0.95 (threshold in `config.yaml`)
 - Significant data drift is detected in key volatility features (Evidently threshold: 0.05)

Versioning & Reproducibility

Model Version: 1.0

Training Script: `models/train.py`

Inference Script: `models/infer.py`

Configuration: `config.yaml` (project root)

Random Seed: 42

Key Dependencies:

- `scikit-learn==1.7.2`
- `pandas==2.3.3`
- `mlflow==3.5.1`
- `numpy==2.3.4`

Configuration Management: The project uses a hybrid configuration approach:

- **`config.yaml`:** Contains all default parameters (feature windows, model hyperparameters, thresholds)
- **Environment variables:** Override defaults for deployment (see `.env.example`)
- **Key parameters documented in:** `docs/feature_spec.md`

Reproducibility: The entire training and evaluation pipeline can be reproduced by:

1. Review configuration: `cat config.yaml`
2. Set environment variables (optional): `cp .env.example .env`
3. Run training: `python models/train.py --features data/processed/features.parquet`
4. All experiments logged to: `mlruns/` directory
5. Model artifacts saved with versioned dependencies

Contact & Support

Owner: Asli Gulcur - CMU Heinz - Operationalizing AI Course

Last Updated: November 8, 2025

Model Status:  **PRODUCTION READY**

Risk Level:  Low (model exceeds all performance thresholds with robust validation on test data)